

EXPERIMENT 1

Aim: Cryptography in Blockchain, Merkle root Tree Hash

Theory :

Cryptography is one of the fundamental technologies used in Blockchain to provide security, data integrity, authentication, and immutability. A blockchain uses cryptographic hash functions to convert input data into a fixed-length string called a hash. In this experiment, the **SHA-256 (Secure Hash Algorithm-256) cryptographic hash function** is used. SHA-256 produces a 256-bit hash, represented as 64 hexadecimal characters. Even a small change in the input produces a completely different hash, making it useful for detecting data modification.

A **Cryptographic Puzzle** is a computational problem used in blockchain systems such as Bitcoin to implement Proof of Work (PoW). Miners repeatedly change a value called a nonce and calculate the hash of the data combined with the nonce. The objective is to find a hash satisfying a specific condition, such as having a required number of leading zeros. The nonce that successfully satisfies the condition is called the Golden Nonce. This process requires computational effort and helps secure the blockchain against fraudulent modifications.

A **Merkle Tree** is a binary tree structure used to efficiently store and verify the integrity of transactions in a blockchain block. Each transaction is first converted into a cryptographic hash. These transaction hashes are then combined in pairs and hashed again to create parent hashes. The process continues until only one hash remains at the top, called the Merkle Root.

If the number of hashes at any level is odd, the last hash is generally duplicated so that all hashes can be paired. The Merkle Root acts as a single cryptographic summary of all transactions in the block. If even one transaction is changed, its hash changes, which causes the subsequent parent hashes and ultimately the Merkle Root to change.

Merkle Trees provide several benefits, including efficient transaction verification, data integrity, reduced storage requirements, and fast detection of changes. They allow a transaction to be verified without downloading or checking every transaction in the block. Merkle Trees are widely used in Bitcoin and other blockchain systems for transaction verification and maintaining the integrity of block data.

Thus, cryptographic hashing, Proof of Work, and Merkle Trees together provide important mechanisms for achieving security, integrity, immutability, and efficient verification in blockchain technology.

Tasks Performed:**1. Hash Generation using SHA-256:**

- o Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.

PROGRAM :

```
import hashlib
data = input("Enter the data: ")
hash_value = hashlib.sha256(data.encode()).hexdigest()
print("SHA-256 Hash:", hash_value)
```

OUTPUT :

Enter the data: HARSHAL CHUGWANI

SHA-256

Hash :

6a2a01f31fa61b77aed366e0427b993531bc9108740d8fb186359e2663dea4d3

2. Target Hash Generation with Nonce:

- o Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.

PROGRAM :

```
import hashlib

data = input("Enter the data: ")
nonce = int(input("Enter the nonce: "))
combined_data = data + str(nonce)

hash_value = hashlib.sha256(combined_data.encode()).hexdigest()

print("Combined Data:", combined_data)
print("SHA-256 Hash:", hash_value)
```

OUTPUT :

Enter the data: HARSHAL CHUGWANI

Enter the nonce: 100

Combined Data: HARSHAL CHUGWANI100

SHA-256

Hash:

eb42b03b3813c09632c87922991468be245ac29dc3d96b205c4dd2e87752d4db

3. Proof-of-Work Puzzle Solving:

- o Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.

PROGRAM:

```
import hashlib
data = input("Enter the data: ")
difficulty = int(input("Enter difficulty (number of leading
                        zeros): "))
target = "0" * difficulty
nonce = 0
while True:
    combined_data = data + str(nonce)
    hash_value = hashlib.sha256(combined_data.encode()).hexdigest()
    if hash_value.startswith(target):
        break
    nonce += 1

print("\nGolden Nonce:", nonce)
print("Hash:", hash_value)
print("Attempts:", nonce + 1)
```

OUTPUT:

Enter the data: HARSHAL CHUGWANI

Enter difficulty (number of leading zeros): 4

Golden Nonce: 35791

Hash:

000071a2465233ae6595ce35c5c3ce7ed7b629d6710791079652c6531f13d854

Attempts: 35792

4. Merkle Tree Construction:

- o Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

PROGRAM:

```
import hashlib
def sha256(data):
    return hashlib.sha256(data.encode()).hexdigest()
transactions = [
    "Alice -> Bob : $200",
    "Bob -> Dave : $500",
    "Dave -> Eve : $100",
    "Eve -> Alice : $300",
    "Roo -> Bob : $50"
]
print(" TRANSACTION HASHES \n")
hashes = []

for transaction in transactions:
    transaction_hash = sha256(transaction)
    hashes.append(transaction_hash)
    print("Transaction:", transaction)
    print("Hash      :", transaction_hash)
    print()

level = 1
while len(hashes) > 1:
    print(" LEVEL", level)
    new_hashes = []
    if len(hashes) % 2 != 0:
        hashes.append(hashes[-1])
    for i in range(0, len(hashes), 2):
        left = hashes[i]
        right = hashes[i + 1]
        combined = left + right
        parent_hash = sha256(combined)
        new_hashes.append(parent_hash)
        print("Left Hash :", left)
        print("Right Hash :", right)
```

```
    print("Parent Hash:", parent_hash)
    print()
    hashes = new_hashes
    level += 1
merkle_root = hashes[0]
print("MERKLE ROOT HASH:")
print(merkle_root)
```

OUTPUT:

TRANSACTION HASHES

Transaction: Alice -> Bob : \$200

Hash :

f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760

Transaction: Bob -> Dave : \$500

Hash :

768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46

Transaction: Dave -> Eve : \$100

Hash :

43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da

Transaction: Eve -> Alice : \$300

Hash :

cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a

Transaction: Roo -> Bob : \$50

Hash :

cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac

LEVEL 1

Left Hash:

f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760

Right Hash:

768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cddb5b46

Parent Hash:

471608d3d6f2a642ff3a84dfaeeaebb1c271a9aed5b37a82d61c4784558e80e9

Left Hash:

43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da

Right Hash:

cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a

Parent Hash:

a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7

Left Hash:

cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac

Right Hash:

cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac

Parent Hash:

314ca746b3dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e

LEVEL 2

Left Hash:

471608d3d6f2a642ff3a84dfaeeaebb1c271a9aed5b37a82d61c4784558e80e9

Right Hash:

a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7

Parent Hash:

ae847393c3dffa60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee

Left Hash :

314ca746b3dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e

Right Hash :

314ca746b3dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e

Parent Hash:

6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45

LEVEL 3

Left Hash :

ae847393c3dffa60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee

Right Hash :

6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45

Parent Hash:

96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0

MERKLE ROOT HASH:

96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0

Conclusion:

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- **SHA-256 Hashing** provides a secure and unique fingerprint for any input, ensuring data integrity.
- **Nonce and Target Hashing** allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
- **Proof-of-Work Puzzle Solving** shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
- **Merkle Tree Construction** efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.